

自己紹介

志望職種：ゲームプログラマー

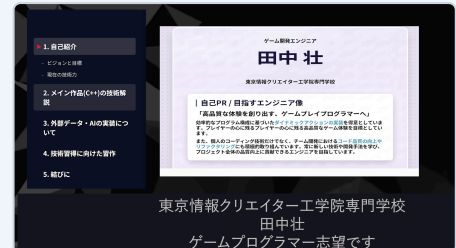
氏名：田中 壮（たなか そう）

所属：東京情報クリエイター工学院専門学校

1. 目指すエンジニア像

プレイヤーが快適に操作できるアクション挙動の実装と、長期的な開発に耐えうる保守性の高いコード設計を両立できるプログラマーを目指しています。

個人の実装速度だけでなく、処理負荷やメモリ消費を考慮したアルゴリズムの選定、チームメンバーが調整しやすいデータ構造の構築を常に意識して制作に取り組んでいます。



2. 保有技術

C++ / DXライブラリ

C++17 DXライブラリ Effekseer Visual Studio 2022

クラス設計、メモリ管理、自作の当たり判定・物理挙動計算、エフェクト制御の実装経験。

C# / Unity

C# Unity

コンポーネント指向による設計、3D空間上のオブジェクト制御、UI実装。

Python

Python

ゲームロジックの構築、2D描画処理、動作検証用の試作制作。

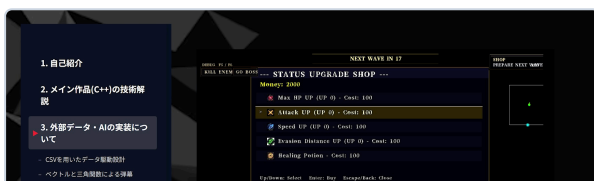
Web技術

PHP SQL JavaScript HTML/CSS

クライアントからの通信受信、データベースへのスコア保存、ランキング一覧の表示構築。

3. 応用技術・基礎習得

外部データによるパラメータ調整



敵の出現パターンやギミック配置、アイテム数値をCSVファイルに外部化し、プログラムの再

三角関数とベクトルによる弾幕制御

角度計算とベクトル演算を組み合わせ、2D作品において規則的で視覚効果の高い弾幕パターンを実装。

主な制作実績①『も～たまん爽快牛キャッチャー』

5名チーム制作 | 3Dアクション |
C++ / DxLib

【プロジェクト概要】

商用ゲームエンジンを使用せず、C++とDxLibを用いてゼロから構築したフル3Dスコアアタックゲーム。UFOを操作して地上にいる牛をビームで捕獲し、ゲージ管理と連続捕獲によるコンボ維持を駆使してハイスコアを目指します。

担当役割：チームリーダー、メインプログラマー、企画補助

技術的な取り組みと成果

1. 共通基盤（アーキテクチャ）の構築

5人体制の開発において、他メンバーの実装が円滑に進むよう明確な責任分離を設計。

- ・SceneManagerによるライフサイクルの抽象化。
- ・遅延削除を採用したObjectManagerによるイテレーション中の不正アクセス防止。
- ・ServiceLocatorによる依存分離を行い、変更に強い設計を確立しました。

2. 自作当たり判定システム（数学的実装）

ゲームエンジンに頼らず、3D空間における衝突計算をゼロから数学的に実装。

- ・スフィア判定：2点間の距離の2乗と半径の和の2乗を比較し、平方根計算を省略して高速化。
- ・カプセル判定：線分と球の最近接点を算出し、ビーム照射の円柱領域や牛の体形に即した正確な判定を実現。

3. イベント駆動型の衝突検知

単なる接触の真偽値判定にとどまらず、前フレームの接触状態との差分から3種類のイベントを自動発火する仕組み（ColliderManager）を構築。

- ・OnEnter（効果音、初回ヒット）
- ・OnTrigger（連続吸引、エフェクト追従）
- ・OnExit（吸引解除、落下処理）

4. AI制御と堅牢なコード品質管理

・Stateパターン：牛の各行動（待機・逃走・吸引等）を独立した状態クラスとして実装し、疎結合なAIを構築。

- ・品質担保：const修飾子やconst参照渡しを徹底し不慮の書き換えを防止。Doxygenによるドキュメント整備を行い、Release x64環境にて0エラー・0警告を達成。

【プロジェクト概要】

ゲームエンジンに頼らず、基幹システムから自力で設計・実装した個人制作の3Dアクション作品。プレイヤーへの心地よい操作感の提供と、拡張性を意識した基盤作りに注力しました。



技術的な取り組みと成果

1. 状態管理（ステートマシン）の設計

キャラクターの行動（待機・移動・回避・各種攻撃）が増加した際の複雑化を防ぐため、状態遷移を一括管理する構造を採用。

プレイヤーと敵キャラクターの行動ロジックを共通の基盤として設計し、入力受付やアニメーション終了に応じた遷移を制御することで、新しい攻撃パターンの追加を安全かつ容易に行える構成にしました。

2. 物理演算を用いたジャンプ挙動

敵キャラクターが目標地点（プレイヤー位置）へ不自然な直線補間で移動するのを防ぐため、放物線を描いて正確に着地する挙動を実装。

重力加速度と打ち上げ初速から滞空時間を逆算し、相手との距離に応じた前進速度を動的に算出。距離が離れていても狙い通りの地点に着地する、説得力のあるジャンプ攻撃を実現しました。

3. 処理負荷とメモリ消費の削減

- ・リソース管理：管理クラス経由でハンドルを使い回す仕組みを構築し、ロード時間とVRAM消費を抑制。

- ・オブジェクト再利用：弾や敵の生成・破棄による負荷（フレーム落ち）を防ぐため、インスタンスを再利用する仕組みを導入。

- ・探索の省略：更新があった場合のみリストを再構築しCPU負荷を削減。

4. 当たり判定の統一設計

球やカプセルなど異なる形状の判定処理を統一的に扱うアーキテクチャを設計。

当たり判定の基底クラスに「触れた瞬間」「触れている間」「離れた瞬間」の判定ロジックを集約し、管理クラスから一括で更新。Unityのコライダーに近い使い勝手をC++上で再現し、ステージやキャラクターへの判定設定を容易にしました。

結び

プレイヤーへの心地よい操作感の提供と、チーム開発を見据えた保守性の高い設計を何よりも大切にしています。

リーダーとしての基盤整備・品質管理の経験や、個人制作を通じて培った低レイヤー寄りのメモリ管理、物理挙動の実装経験を活かし、入社後も基礎原理の理解を怠らず、チーム全体の開発効率と製品品質の向上に貢献いたします。

